The master sends messages at the specified rate to a random worker and expects a response within three seconds. This response can be a `Success` with the aggregation of words or a `Failure` with the original message, which is then forwarded to a queue (called `lag`) for reprocessing. If successful, the master merges the aggregation with its current state and keeps the total sum of all aggregations received up to that time. Again, this is shown in figure 8.11.

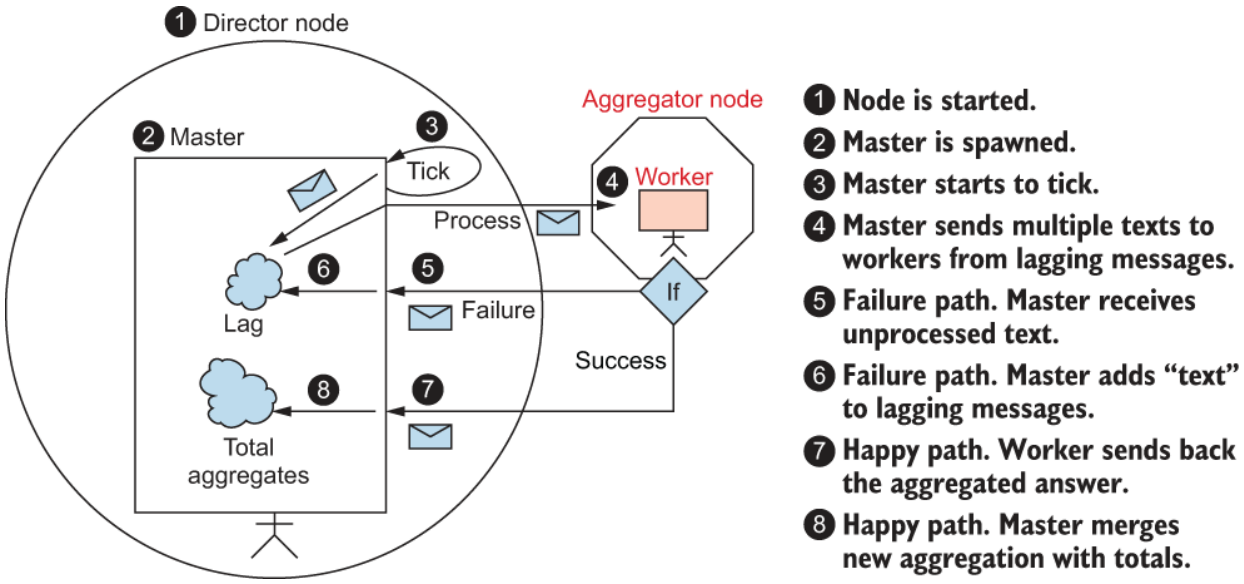


Figure 8.11 Flow of a master

When an aggregator node starts, it creates the number of workers specified in the configuration. These workers register during setup with a `Worker.RegistrationKey` so the master's router, which subscribes to that key, can find them and send them work. From that point on, messages arrive at the workers and are processed, and the results are returned in the form of a `Map[String,Int]` indicating the frequency per word.